fn conservative_continuous_gaussian_tail_to_alpha

Michael Shoemate (@Shoeboxam)

November 13, 2026

Proof for conservative_continuous_gaussian_tail_to_alpha.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `scale` is of type `RBig`
- Argument `tail` is of type `RBig`

Caller-verified

None

Pseudocode

```
def conservative_continuous_gaussian_tail_to_alpha(scale: RBig, tail: RBig) -> f64:
    check_tail_arguments(scale, tail)
    # the SQRT_2 constant is already rounded down
    sqrt_2_ceil = SQRT_2.next_up_()

    # tail and scale division should be big rationals for precision and to avoid overflow
    t = f64.neg_inf_cast(tail / scale).neg_inf_div(sqrt_2_ceil)
    # round down to nearest smaller f32
    t = f64(f32.neg_inf_cast(t))
    # erfc error is at most 1 f32 ulp (see erfc_err_analysis.py)
    t = f32.inf_cast(erfc(t)).next_up_()

    return f64(t).inf_div(2.0)
```

Postcondition

Definition 1.1. Define $X \sim \mathcal{N}(0, \texttt{scale})$, where $\texttt{scale} > 0$. Then X follows the continuous gaussian distribution:

$$f(x) = \frac{1}{\texttt{scale}\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x}{\texttt{scale}}\right)^2} \tag{1}$$

Definition 1.2. The error function is defined as:

$$\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt \tag{2}$$

Definition 1.3. The complementary error function is defined as:

$$\text{erfc}(z) = 1 - \text{erf}(z) \tag{3}$$

Lemma 1.4. The implementation of `erfc` differs from a conservatively rounded implementation by no greater than one 32-bit float ulp.

Proof. The following code conducts an exhaustive search.

```python
# The result from this check is that the erfc function in statrs errs by at most 1 f32 ulp.

# First disagreement is at 0.5.
# Execution is slowest at inputs around 15, before switching to the next approximating curve.

# To use all CPUs, floats are sharded modulo the number of CPUs (less two).
# It may be necessary to restart this program at a later float to free memory.

import struct
import multiprocessing

# pip install gmpy2
import gmpy2
from opendp._data import erfc

# specifically check max ulp distance from a conservative upper bound
gmpy2.get_context().round = gmpy2.RoundUp


def floatToBits(f):
    s = struct.pack(">f", f)
    return struct.unpack(">l", s)[0]


def bitsToFloat(b):
    s = struct.pack(">l", b)
    return struct.unpack(">f", s)[0]


def worker(offset, step):
    max_err = 0
    print(f"running {offset}")
    # iterate through all 32-bit floats
    for bits in range(floatToBits(0.0), floatToBits(float("inf")), step):
        bits += offset
        if offset == 0 and bits > 0 and (bits // step) % 10_000 == 0:
            prop_done = bits / floatToBits(float("inf"))
            print(
                f"{prop_done:.2%} done, with max discovered f32 ulp error of: {max_err}. Currently at: {bitsToFloat(bits)}"
            )

        f32 = bitsToFloat(bits)
        f32_ulp_err = abs(floatToBits(erfc(f32)) - floatToBits(gmpy2.erfc(f32)))
        max_err = max(max_err, f32_ulp_err)
    print(max_err)


if __name__ == "__main__":
    n_cpus = multiprocessing.cpu_count() - 2
    processes = []
    for cpu in range(n_cpus):
        p = multiprocessing.Process(target=worker, args=(cpu, n_cpus))
        p.start()
        processes.append(p)

    [p.join() for p in processes]
```

Upon completion, the greatest discovered error is at most 1 ulp. □

Lemma 1.5. Assume $X \sim \mathcal{N}(0, \texttt{scale})$ and $\texttt{tail} \geq 0$. Then

$$\alpha = P[X > \texttt{tail}] = \frac{1}{2}\text{erfc}\left(\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}\right) \tag{4}$$

Proof. The strict postcondition $P[X > \texttt{tail}]$ is intentional, because dependent proofs use this exact event. The argument in this theorem is about the ideal continuous gaussian random variable X. For this reference distribution, continuity implies $P[X > \texttt{tail}] = P[X \geq \texttt{tail}]$. This equality is used only within the theorem, and not for any discretized or postprocessed implementation, where $P[X > \texttt{tail}]$ and $P[X \geq \texttt{tail}]$ may differ due to point masses.

$$\begin{aligned}
\alpha &= P[X > \texttt{tail}] \\
&= P[X \geq \texttt{tail}] && \text{because } X \text{ is continuous} \\
&= \int_{\texttt{tail}}^{\infty} \frac{1}{\texttt{scale}\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x}{\texttt{scale}}\right)^2} dx && \text{by 1.1} \\
&= \frac{1}{\sqrt{\pi}} \int_{\texttt{tail}/(\texttt{scale}\sqrt{2})}^{\infty} e^{-u^2} du && \text{with } u = \frac{x}{\texttt{scale}\sqrt{2}} \\
&= \frac{1}{2}\left(1 - \text{erf}\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}\right) && \text{by 1.2} \\
&= \frac{1}{2}\text{erfc}\left(\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}\right) && \text{by 1.3}
\end{aligned}$$

□

Theorem 1.6. For any inputs `scale` and `tail` satisfying the preconditions, `conservative_continuous_gaussian_tail_to_alpha` either:

- returns `Err(e)`, or
- returns `Ok(out)` with $\texttt{out} \geq \Pr[X > \texttt{tail}]$, where $X \sim \mathcal{N}(0, \texttt{scale})$.

Proof. `check_tail_arguments` returns `Err(e)` unless $\texttt{scale} > 0$ and $\texttt{tail} \geq 0$. Otherwise, if no remaining operation returns `Err(e)`, the implementation computes $\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}$ with conservative downward rounding into a non-negative 32-bit float, so the input to `erfc` is no greater than the exact value. Since `erfc` is monotonically decreasing, this yields a value no smaller than $\text{erfc}\left(\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}\right)$. The outcome of `erfc` is then rounded up and increased by one 32-bit float ulp, which compensates for implementation error by Lemma 1.4, and divided by 2.0 with conservative upward rounding. Therefore the returned result is at least $\Pr[X > \texttt{tail}]$ by Lemma 1.5. □